בס"ד

## מערכות הפעלה – בית ב'

1. הבעיה יסודית שגורמת לאי עקביות בקבצי היומן ובקובץ החשבון היא חוסר בסנכרון נכון בין הסוכנים שמעדכנים את המשאבים השותפים. בסביבה מקבילה, סוכנים מרובים מנסים לקרוא ולכתוב לקובץ החשבון בו זמנית.
נשתמש בסמפור בינארי כדי לגשת סינכרונית לקובץ החשבון.
נתחיל עם הוספת הסמפור וההתחול שלו בקובץ agents.c,
נעדכן את הפונקציות update ו- go_agent כך שישתמשו בסמפור.

2. הבקר מבצע בדיקות עקביות של קבצי היומן וקובץ החשבון במרווחים קבועים או בנקודות ספציפיות בתוכנית .כדי להפוך את הבדיקות לאקראיות, נשתמש במחולל מספרים אקראיים כדי לקבוע את הרווחים, בבקר יבצעו את הבדיקות.

3. לרוב שימוש בפונקציה הנ"ל בסוף ה- main הוא רעיון טוב, אומנם צריך לשים לב שכל התהליכונים סיימו את עבודתם לפני שפונ' main הסתיימה.

## הקוד שעונה על המשימות:

hodaya2002@LAPTOP-SRUSS0I8: ~

GNU nano 6.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <errno.h>
#include <pthread.h>
#include <semaphore.h>
#include "agents.h"

// Global variables
char agents[MAX_AGENTS][NAMES_LEN] = AGENTS;
int p;
pthread_t thread[MAX_AGENTS], cThread;
sem_t mutex;

// Function declarations
int init(int num_agents);
int check_point(int *num_agents);
void go_agent(char *agent_name);
int log_sum(char *log_file);
int update(int amount, char *account, char *name);
int write_to_log(int amount, char *log_file);
void fatal_error(char *err_msg, char *perror_arg);
int gen_rand(int range);

int main(int ac, char **av) {
    int i = 0, num_agents = 1;

    // Check command-line arguments
    if (ac == 2)
        i = atoi(av[1]);
    else if (ac != 1)
        fatal_error("USAGE: agents [number_of_agents]\n", NULL);

    // Initialize semaphores
    sem_init(&mutex, 0, 1);

    // Initialize agents
    num_agents = init(i);

    // Create threads for agents
    for (i = 0, p = 0; i < num_agents; i++, p++) {
        if (pthread_create(&thread[p], NULL, (void *)go_agent, &agents[p])) {
            perror("ERROR creating thread.");
            --p;
```

GNU nano 6.2

```c
        }
    }

    printf("Finished creating %d agent(s). \n", num_agents);
    sleep(1);

    // Wait for agents to finish
    while (p >= 1) {
        sleep(2);
    }

    // Check consistency
    check_point(&i);

    // Wait for all threads to finish
    for (i = 0; i < num_agents; i++) {
        pthread_join(thread[i], NULL);
    }

    // Destroy semaphore
    sem_destroy(&mutex);

    return 0;
}

// Initialize auxiliary files and return number of agents
int init(int num_agents) {
    FILE *fp;
    int i;

    if (num_agents < 1 || num_agents > MAX_AGENTS) {
        printf("%d agent(s) defaults to 1\n", num_agents);
        num_agents = 1;
    }

    printf("Creating %d agent(s). \n", num_agents);

    // Delete old log files if they exist
    for (i = 0; i < num_agents; i++) {
        if ((unlink(agents[i]) < 0) && (errno != ENOENT)) {
            fatal_error("Deleting agents log", "");
        }
    }

    // Create new account file
    if ((fp = fopen(ACCOUNT, "w")) == NULL) {
```

```c
    // Create new account file
    if ((fp = fopen(ACCOUNT, "w")) == NULL) {
        fatal_error("Creating ACCOUNT file", "");
    }
    fprintf(fp, "%d\n", START_SUM);
    fclose(fp);

    return num_agents;
}

// Check consistency of log files and account file
int check_point(int *num_agents) {
    int i, n, control_sum, sum;

    control_sum = 0;

    // Sleep for random interval
    if ((n = rand() % MAX_SLEEP) != 0) {
        sleep(n * 3);
    }

    printf("\n\n ----- Controller ----- \n");

    // Get sum from account file
    if ((sum = update(0, ACCOUNT, NULL)) == ERROR) {
        return ERROR;
    }

    // Calculate sum from log files
    for (i = 0; i < *num_agents; ++i) {
        control_sum += log_sum(agents[i]);
    }

    control_sum = START_SUM + control_sum;

    // Compare sums
    if (sum != control_sum) {
        fprintf(stderr, "\n Not consistent! Somebody steals?!\n");
        fprintf(stderr, "Controller: %d, Account: %d \n", control_sum, sum);
        fprintf(stderr, " Terminating all agents!\n");
        return 1;
    }

    printf("\nOperations OK. Controller: %d, Account: %d\n", control_sum, sum);

    exit(SUCCESS);
```

בס"ד

```c
// Calculate sum of numbers in log file
int log_sum(char *log_file) {
    FILE *fp;
    int sum = 0, tmp_sum;

    if ((fp = fopen(log_file, "r+")) == NULL) {
        perror(log_file);
        printf("\nCan't Control %s agent!\n", log_file);
        return 0;
    }

    while (fscanf(fp, "%d", &tmp_sum) > 0) {
        sum += tmp_sum;
    }

    fclose(fp);

    printf(" Sum of %s = %d\n", log_file, sum);

    return sum;
}

// Print error message and exit
void fatal_error(char *err_msg, char *perror_arg) {
    fprintf(stderr, "\nERROR: %s. ", err_msg);
    if (perror_arg != NULL) {
        perror(perror_arg);
    }
    exit(ERROR);
}

// Agent thread function
void go_agent(char *agent_name) {
    int n, i, amount;

    for (i = 0; i < OPERATIONS; ++i) {
        amount = gen_rand(MAX_OPER);

        // Update account file
        if (update(amount, ACCOUNT, agent_name) == ERROR) {
            fatal_error("Can't Update Account File", ACCOUNT);
        } else {
            // Write operation to log file
            if (write_to_log(amount, agent_name) == ERROR) {
```

```c
            if (write_to_log(amount, agent_name) == ERROR) {
                if (update(-amount, ACCOUNT, NULL) == ERROR) {
                    fprintf(stderr, "ERROR: Rollback failed. Agent %s\n", agent_name);
                }
                fatal_error("Can't log operations", agent_name);
            }
        }

        printf(" Op of %s = %d\n", agent_name, amount);

        // Sleep for random interval
        if ((n = rand() % MAX_SLEEP) != 0) {
            sleep(n);
        }
    }

    p--;
    printf(" Agent %s : Successful!\n", agent_name);

    return;
}

// Generate random number within range
int gen_rand(int range) {
    int n, sign;
    static int s = 17;

    srand(time(NULL) % s++);

    while ((n = rand()) == 0);

    sign = (n / 1111) % 2;
    n = ((n / 1117) % range) + 1;

    if (sign == 0) {
        return n;
    } else {
        return -n;
    }
}

// Update account file with amount
int update(int amount, char *account, char *name) {
    FILE *fp;
    int n;
    char buf[BUF_LENGTH];
```

```c
    // Read current amount from account file
    if (!(fp = fopen(account, "r"))) {
        return ERROR;
    }

    fscanf(fp, "%s", buf);
    fclose(fp);

    // Update account with new amount
    if (!(fp = fopen(account, "w"))) {
        return ERROR;
    }

    fprintf(fp, "%d", n = atoi(buf) + amount);
    fclose(fp);

    return n + amount;
}

// Write amount to agent's log file
int write_to_log(int amount, char *log_file) {
    FILE *fp;

    if ((fp = fopen(log_file, "a")) == NULL) {
        return ERROR;
    }

    fprintf(fp, "%d\n", amount);
    fclose(fp);

    return SUCCESS;
}
```